CENG 114 BİLGİSAYAR BİLİMLERİ İÇİN AYRIK YAPILAR

Prof. Dr. Tufan TURACI
tturaci@pau.edu.tr

- Pamukkale Üniversitesi
- Mühendislik Fakültesi
- Bilgisayar Mühendisliği Bölümü
- Hafta 14

Ders İçeriği

- **Graf Veri Yapısı**
- **Graflarda Uzaklık Kavramı**

--- Bir grafın çapı, yarıçapı, merkez ve kıyı tepeleri

--- Dijsktra Algoritması

Graf Veri Yapısı

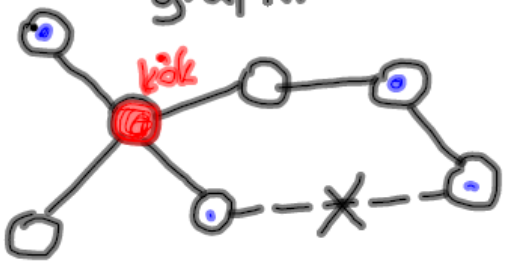
--- Bilgisayar dünyasında bulunan ve gerçek hayatta çeşitli sebeplerle karşılaşılan yapıları temsil etmek amacıyla graflar kullanılır.

--- Örneğin bir bilgisayar ağını, her türlü iletişim problemini graflar ile modelleyebiliriz veya küçükten büyüğe sıralanmış sayıları saklamak için ağaç grafları kullanabiliriz.

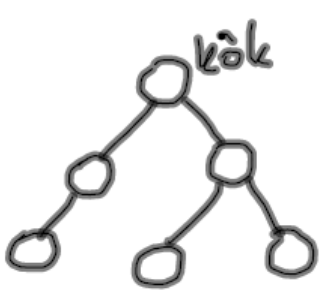
--- Belirli bir kurala göre bir ikili ağaçta dolaşma ve arama için graf veri yapısı kullanılır.

Örnek: İkili Arama Ağacı Binary Search Tree (BST)

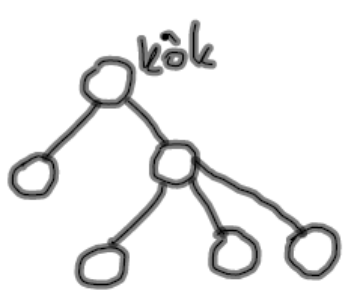
Ağaç: Döngü (çevre, çevrim, cycle) içermeyen bir graf veya Her iki düğümü arasında tam 1 yol içeren graftır.



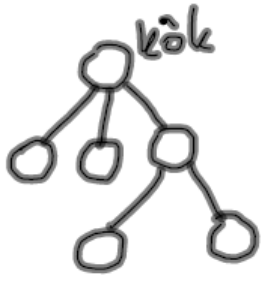
İkili ağaç: (Binary tree): Her düğümün en fazla 2 çocuğu olabilir.



Evet



Hayır



Hayır

İkili Arama Ağacı (Binary Search Tree):

x ikili arama ağacında bir düğüm olsun.

Eğer y düğümü x'in sol alt ağacında ise

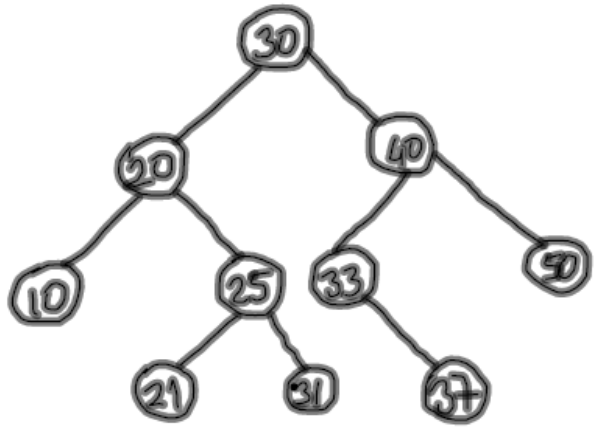
$$key[y] \leq key[x]$$

aksi durumda ise

$$key[y] \geq key[x]$$



BST dir



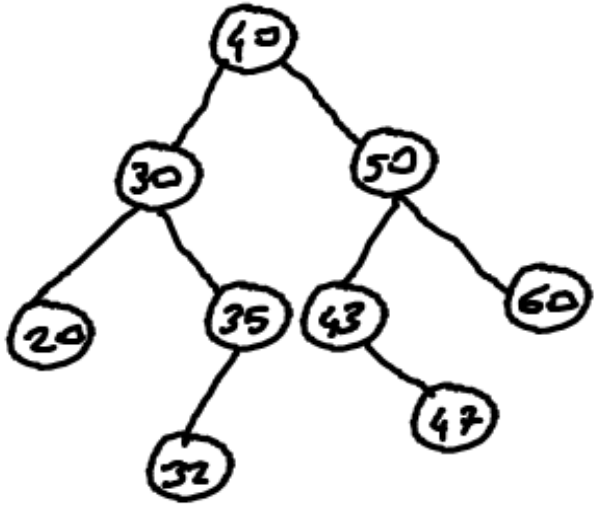
BST değil

İkili Ağaçlarda Dolaşma

Inorder Tree Walk: sol - kök - sağ

Preorder Tree Walk: kök - sol - sağ

Postorder Tree Walk: sol - sağ - kök



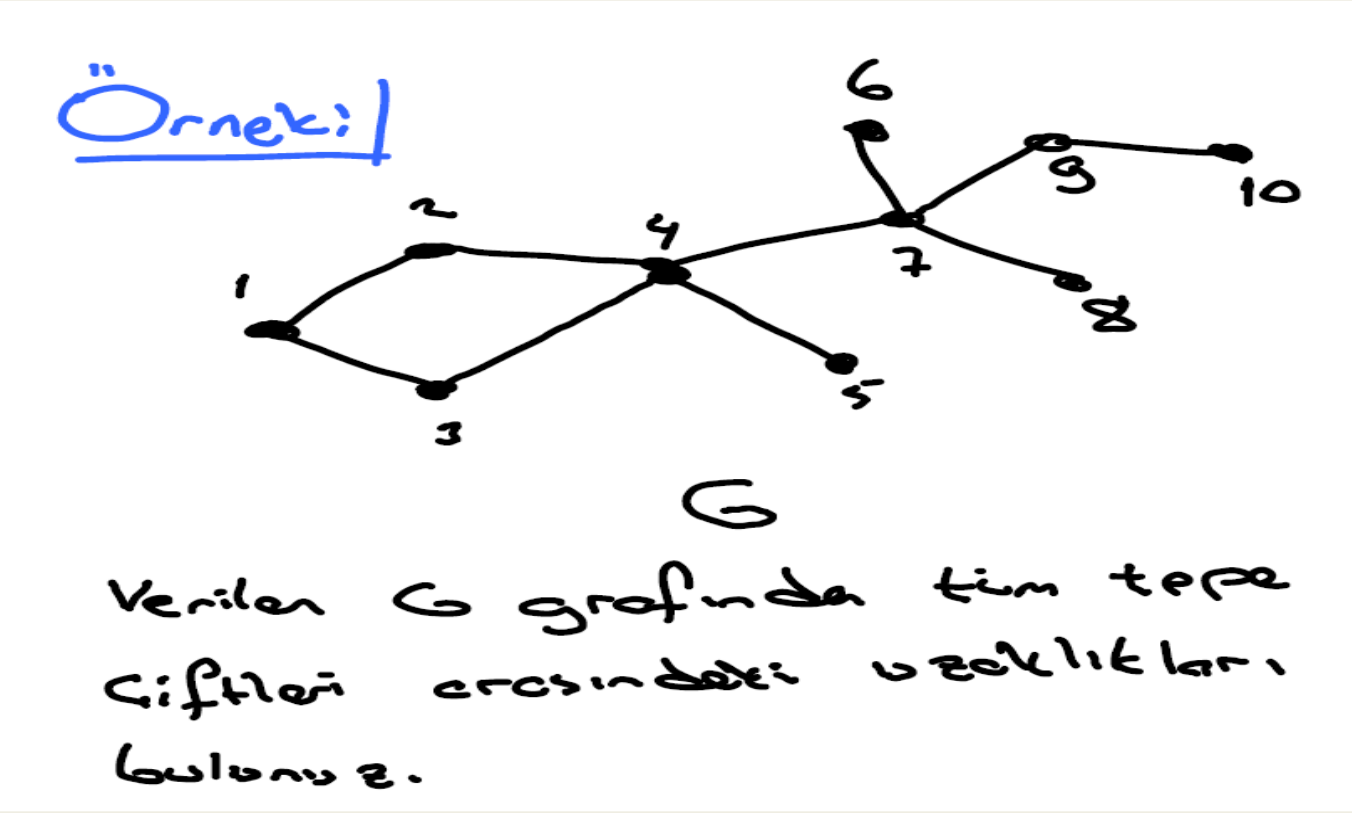
Inorder: 20, 30, 32, 35, 40, 43, 47, 50, 60

Preorder: 40, 30, 20, 35, 32, 50, 43, 47, 60

postorder: 20, 32, 35, 30, 47, 43, 60, 50, 40

Graflarda Uzaklık Kavramı

Tanım: Bir G grafında u ve v gibi iki tepe arasındaki yollar içinde minimum uzunluğu olanın uzunluğuna; u ve v nin **uzaklığı (distance)** denir ve $d(u,v)$ (veya $d_G(u,v)$) biçiminde gösterilir.



$d(1,2) = 1$

$d(1,3) = 1$

$d(1,4) = 2$

$d(1,5) = 3$

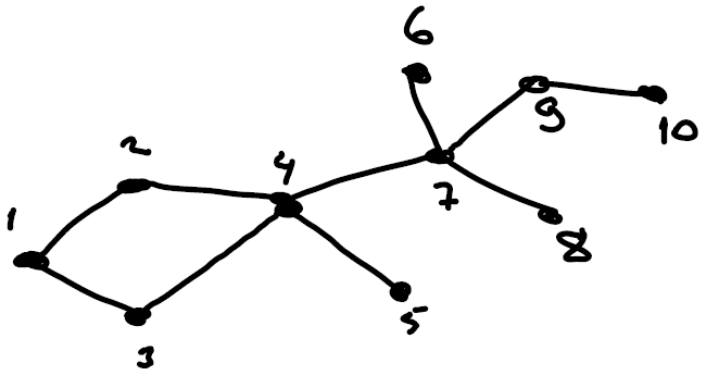
$d(1,6) = 4$

$d(1,7) = 3$

$d(1,8) = 4$

$d(1,9) = 4$

$d(1,10) = 5$



G

$d(2,3) = 2$

$d(2,4) = 1$

$d(2,5) = 2$

$d(2,6) = 3$

$d(2,7) = 2$

$d(2,8) = 3$

$d(2,9) = 3$

$d(2,10) = 4$

$d(3,4) = 1$

$d(3,5) = 2$

$d(3,6) = 3$

$d(3,7) = 2$

$d(3,8) = 3$

$d(3,9) = 3$

$d(3,10) = 4$

$d(4,5) = 1$

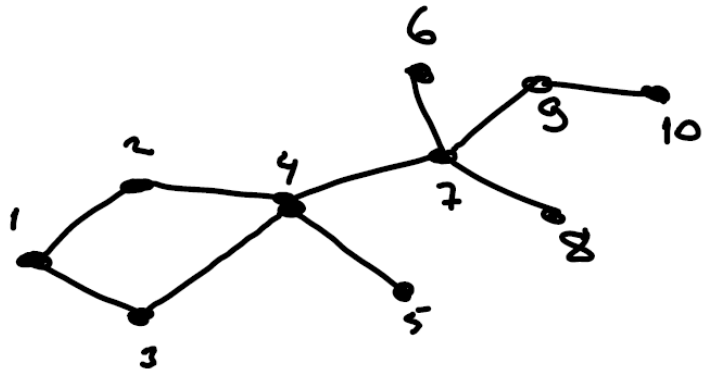
$d(4,6) = 2$

$d(4,7) = 1$

$d(4,8) = 2$

$d(4,9) = 2$

$d(4,10) = 3$



G

$d(5,6) = 3$

$d(5,7) = 2$

$d(5,8) = 3$

$d(5,9) = 3$

$d(5,10) = 4$

$d(7,8) = 1$

$d(7,9) = 1$

$d(7,10) = 2$

$d(8,9) = 2$

$d(8,10) = 3$

$d(9,10) = 1$

$d(6,7) = 1$

$d(6,8) = 2$

$d(6,9) = 2$

$d(6,10) = 3$

Tanım 3: Dışmerkezlilik **(eccentricity)**, her tepenin diğer tepelere olan uzaklıklarının en büyük değeridir ve $e(v)$ (veya $e_G(v)$) biçiminde gösterilir. En büyük dışmerkezlilik değerine **çap (diameter)** denir, $diam(G)$ biçiminde gösterilir. En küçük dışmerkezlilik değerine **yarıçap (radius)** denir ve $r(G)$ biçiminde gösterilir.

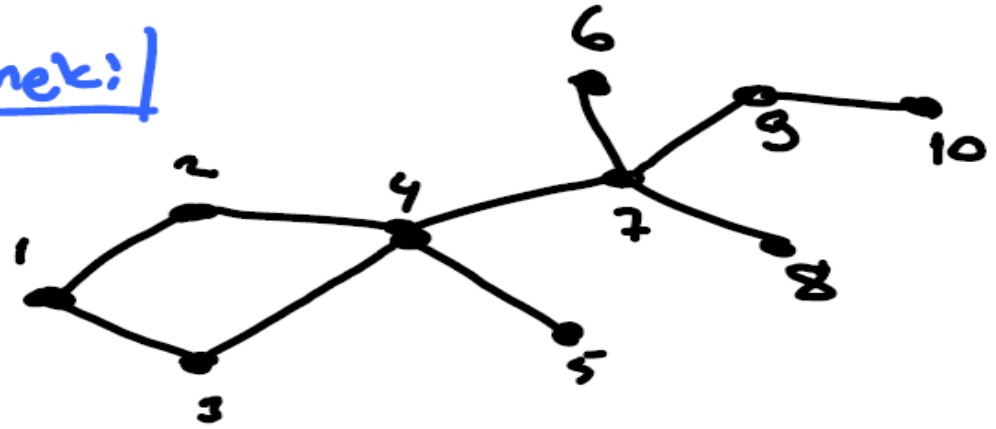
Tanım 4: Dışmerkezlilik değeri yarıçapa eşit olan tepelere **merkez tepeler (central vertices)** denir.

Tanım 5: Dışmerkezlilik değeri çapa eşit olan tepe veya tepelere **kıyı tepeler (peribheral vertices)** denir.

Eccentricity (Dışmerkezlik)

$e(v) = \max \{ d(u,v) \mid u,v \in V(G) \}$

Örnek:



G

$e(1) = 5$

$e(2) = 4$

$e(3) = 4$

$e(4) = 3$

$e(5) = 4$

$e(6) = 4$

$e(7) = 3$

$e(8) = 4$

$e(9) = 4$

$e(10) = 5$

Böylece;

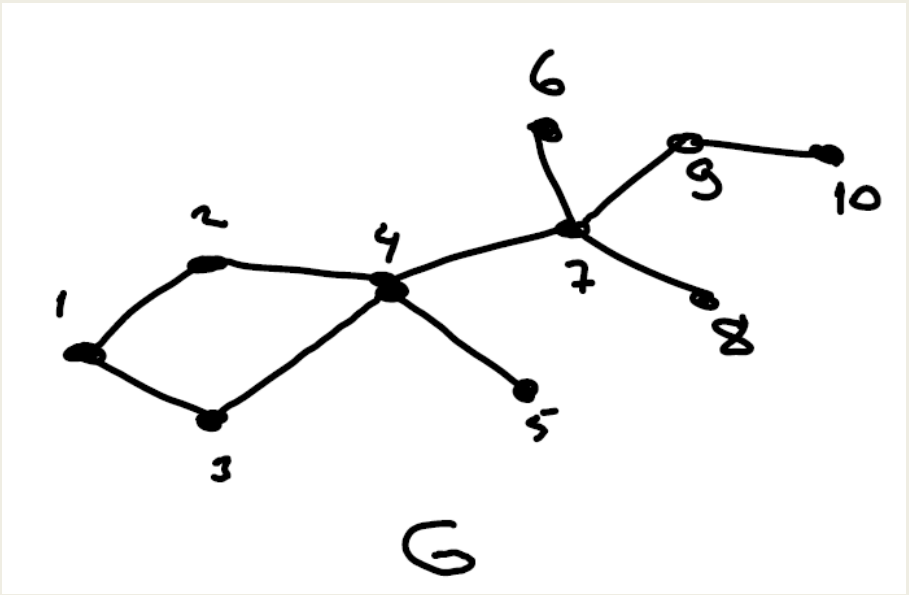
$diam(G) = 5$ (Çap)

$r(G) = 3$ (Yarıçap)

Merkez tepeler;

$C(G) = \{v \mid e(v) = r(G)\}$

$C(G) = \{4, 7\}$



G

Kıyı tepeler;

$p(G) = \{v \mid e(v) = diam(G)\}$

$p(G) = \{1, 10\}$

Dijsktra Algoritması

---Dijkstra algoritması, bir tepeden diğer tüm tepelere en kısa yolları hesaplar.

---Ağırlıklı ve yönlü graflar için geliştirilmiştir. Fakat ağırlıksız ve yönsüz graflar içinde kullanılabilir.

---Dijkstra algoritması en kısa yolları Greedy yaklaşımı ile belirler.

---Yani bir tepeden diğer bir tepeye geçerken olası en iyi yerel çözümü göz önüne alır.

---Her seferinde bir sonraki tepeye ilerleme Greedy yaklaşımına göre yapılır.

Algoritma:

```
Dijkstra_uzaklık (graf G, düğüm v0)
// v0 başlangıç düğüm
{
   S = {v0};
   for(i=1; i<n; i++)
         D[i]=C[v0,i];          // C[i,j] - maliyet matrisi
   for (i=0; i<n-1; i++)
         {
   V-S'den öyle bir w düğümü seçelim ki D[w] minimum olsun;
         w'yi S'ye ekle;

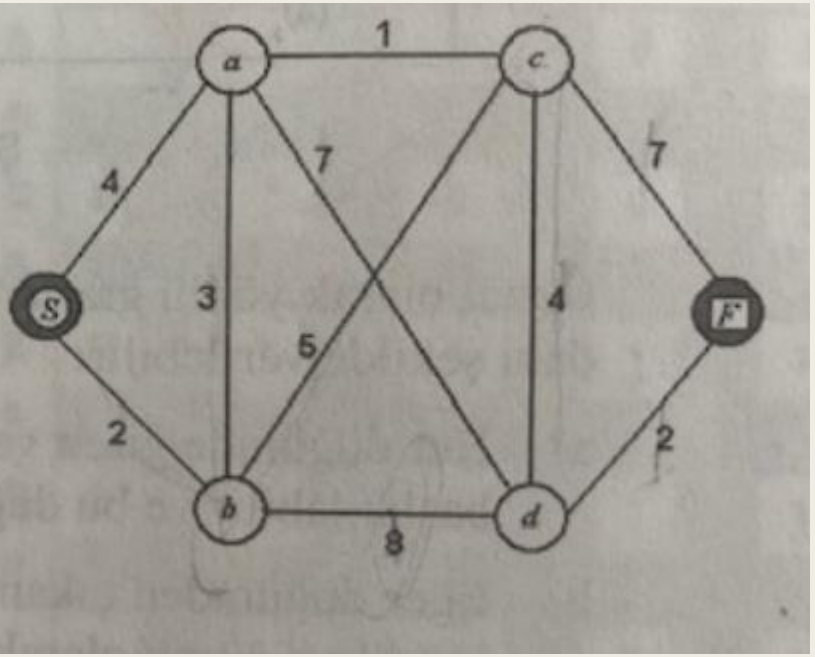
   for v∈V - S
   D[v]=min(D[v],D[w]+C[w,v]);     // uzaklıkları yeniden ayarla
         }
}
```

C kodu çalışma olarak bırakılmıştır!!!

Örnek: Örnek, 5 numaralı kaynaktan alınmıştır.

S den F ye en kısa yolu bulunuz.

A=	S	a	b	c	d	F
S	0	4	2	∞	∞	∞
a	4	0	3	1	7	∞
b	2	3	0	5	8	∞
c	∞	1	5	0	4	7
d	∞	7	8	4	0	2
F	∞	∞	∞	7	2	0



S den uzaklıklar...

Tepeler	S	a	b	c	d	F
Uzaklık	0	4	2	∞	∞	∞

1 . adım: a,b,c,d,F arasından minimum olan b=2

Tepeler	S	a	b*	c	d	F
Uzaklık	0	4	2	∞	∞	∞

(b,a) için $D(a) = \min(4, 2+3) = 4$

(b,c) için $D(c) = \min(\infty, 2+5) = 7$

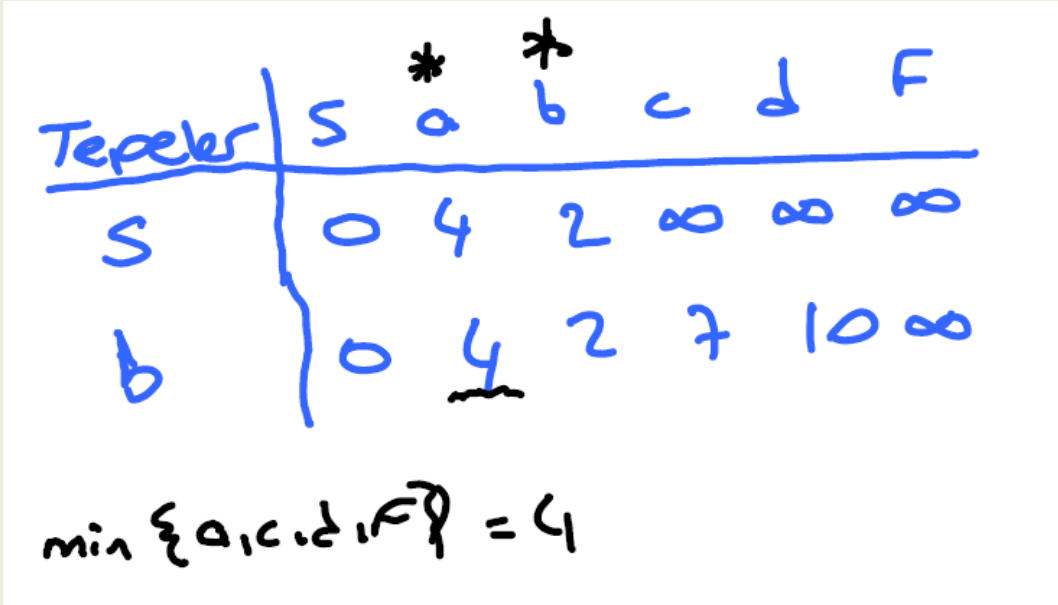
(b,d) için $D(d) = \min(\infty, 2+8) = 10$

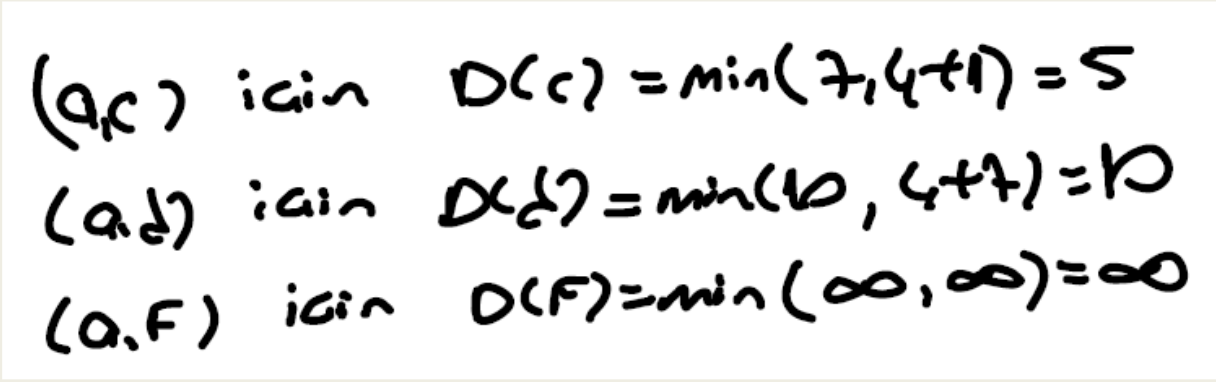
(b,F) için $D(F) = \min(\infty, \infty) = \infty$

Böylece; yeni tablo aşağıdaki gibidir.

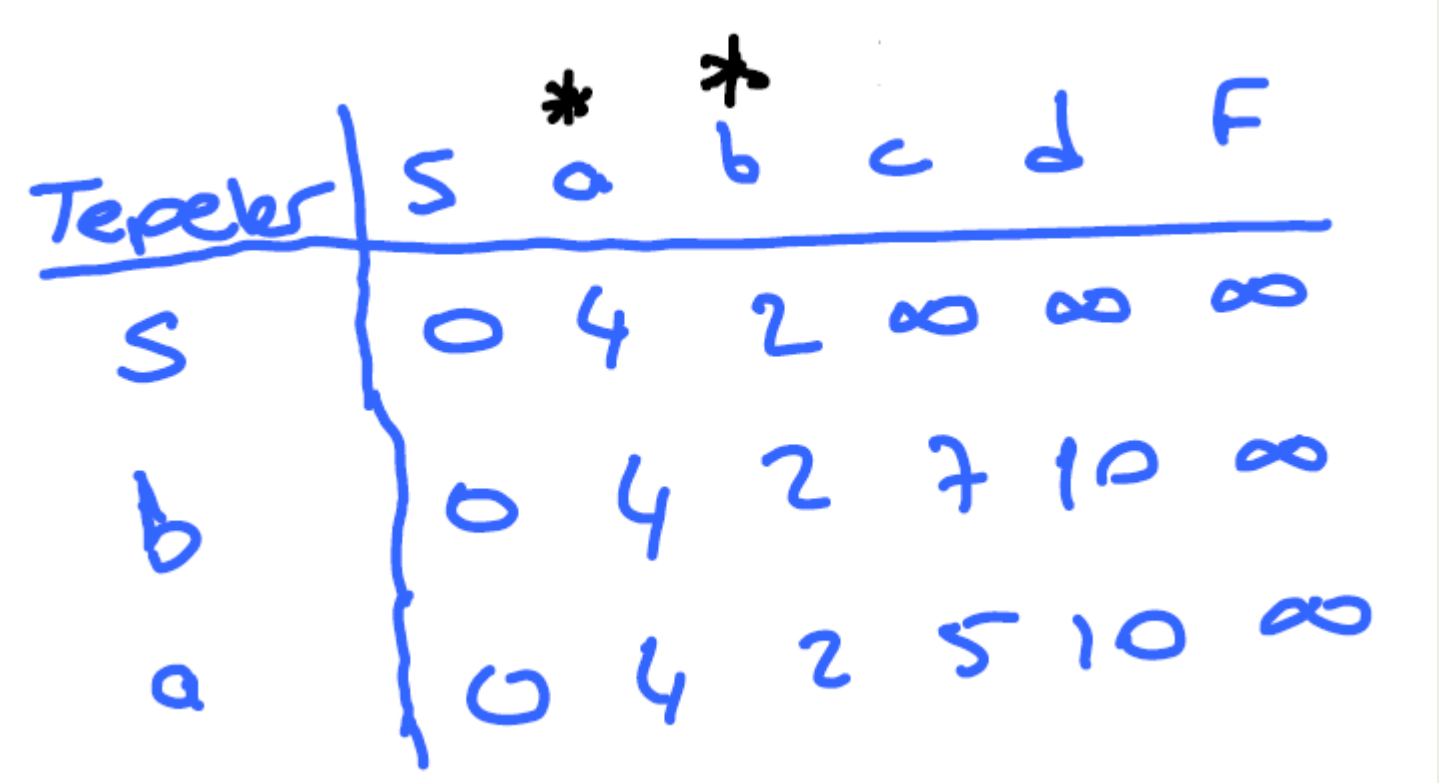
Tepeler	S	a	b*	c	d	F
S	0	4	2	∞	∞	∞
b	0	4	2	7	10	∞

2. adım: a,c,d,F arasından minimum olan
a=4





2. Adım sonrası yeni tablo aşağıdaki gibidir.



Tepeler	S	a*	b*	c	d	F
S	0	4	2	∞	∞	∞
b	0	4	2	7	10	∞
a	0	4	2	5	10	∞

3. adım: c,d,F arasından minimum olan c=5

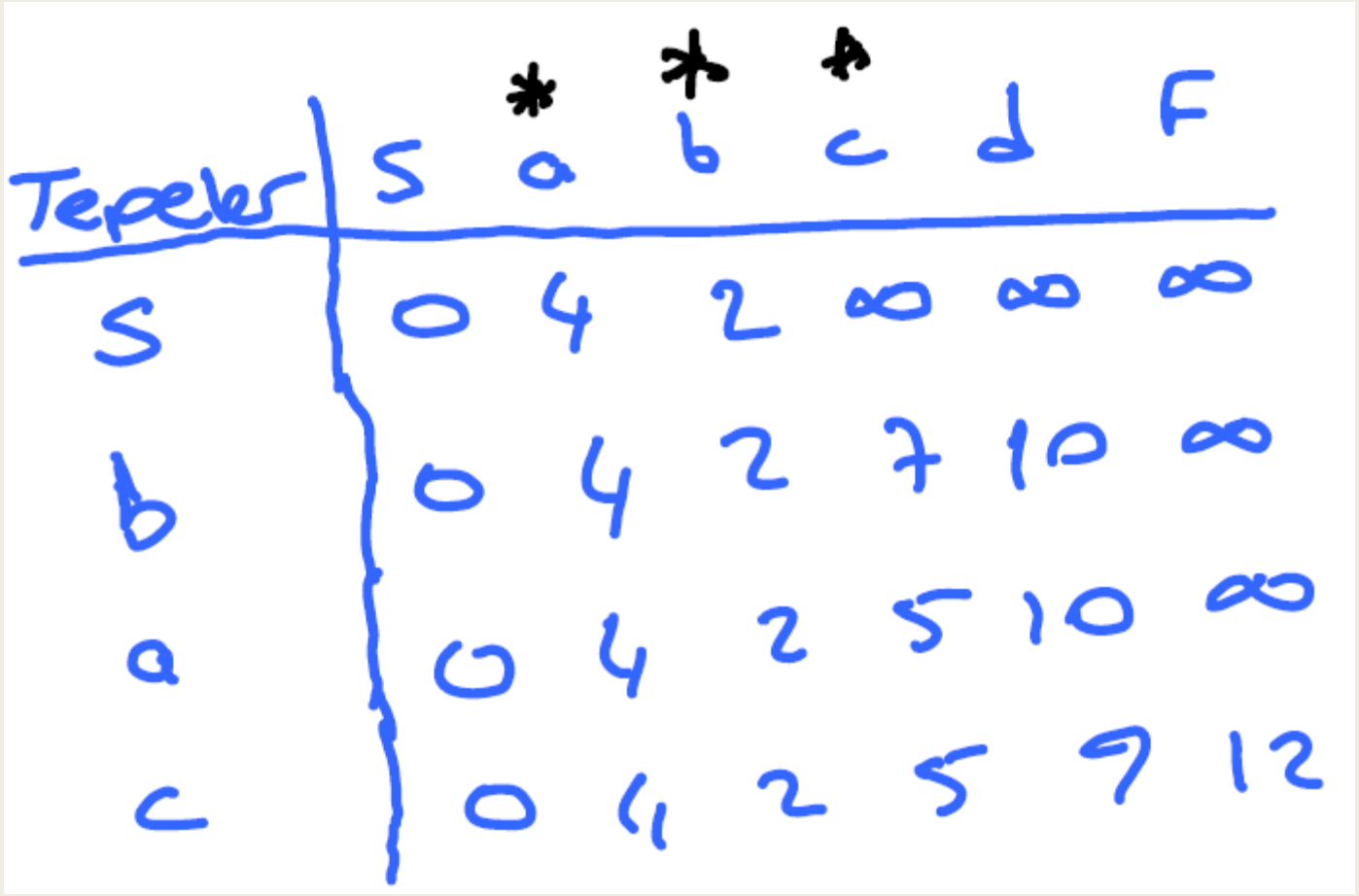
Tepeler	S	a *	b *	c *	d	F
S	0	4	2	∞	∞	∞
b	0	4	2	7	10	∞
a	0	4	2	5	10	∞

$\min\{c, d, F\} = 5$

(c, d) için $D(d) = \min(10, 5+4) = 9$

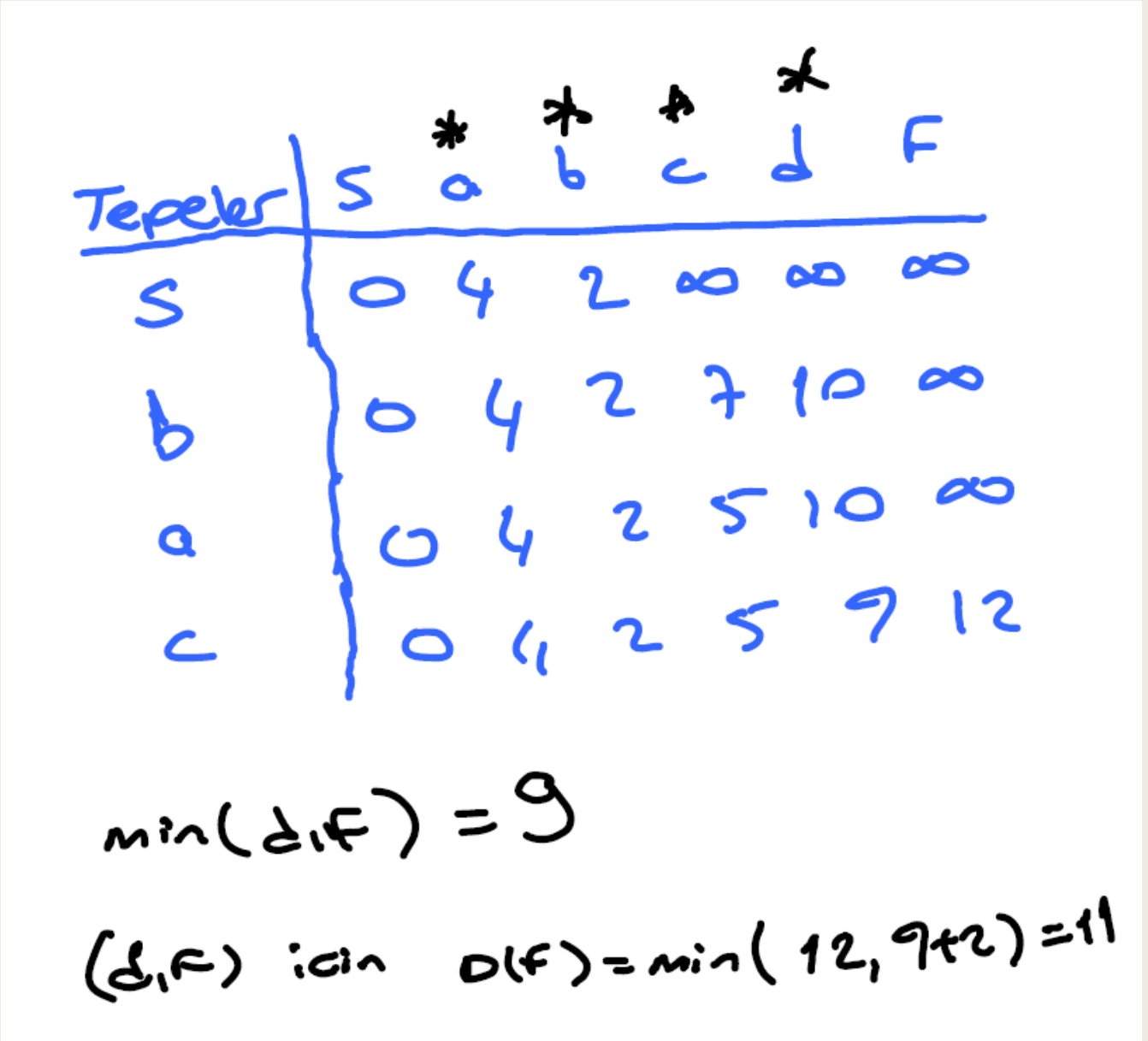
(c, F) için $D(F) = \min(\infty, 5+7) = 12$

3. Adım sonrası yeni tablo aşağıdaki gibidir.

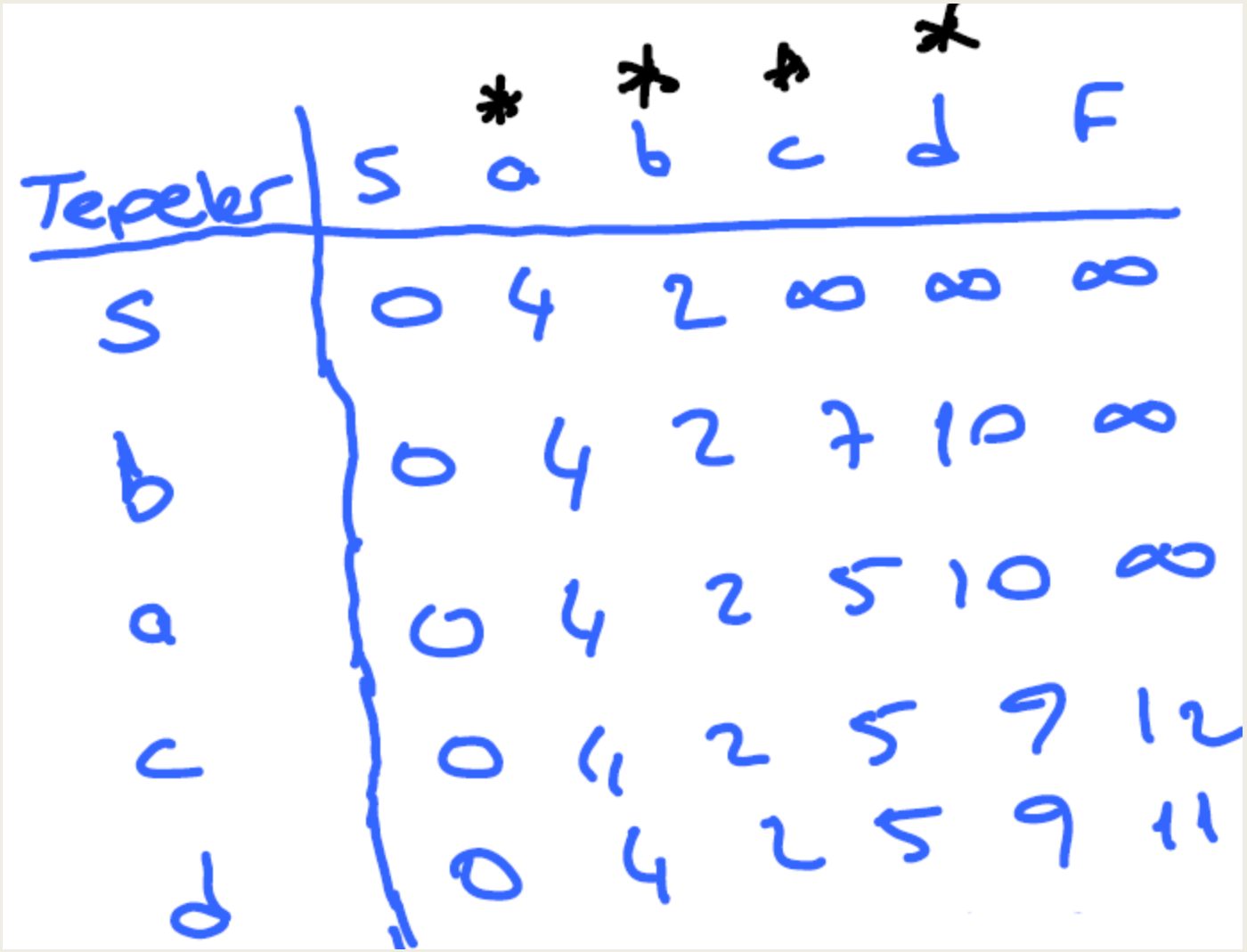


Tepeler	S	a*	b*	c*	d	F
S	0	4	2	∞	∞	∞
b	0	4	2	7	10	∞
a	0	4	2	5	10	∞
c	0	4	2	5	7	12

4. adım: d,F arasından minimum olan d=9

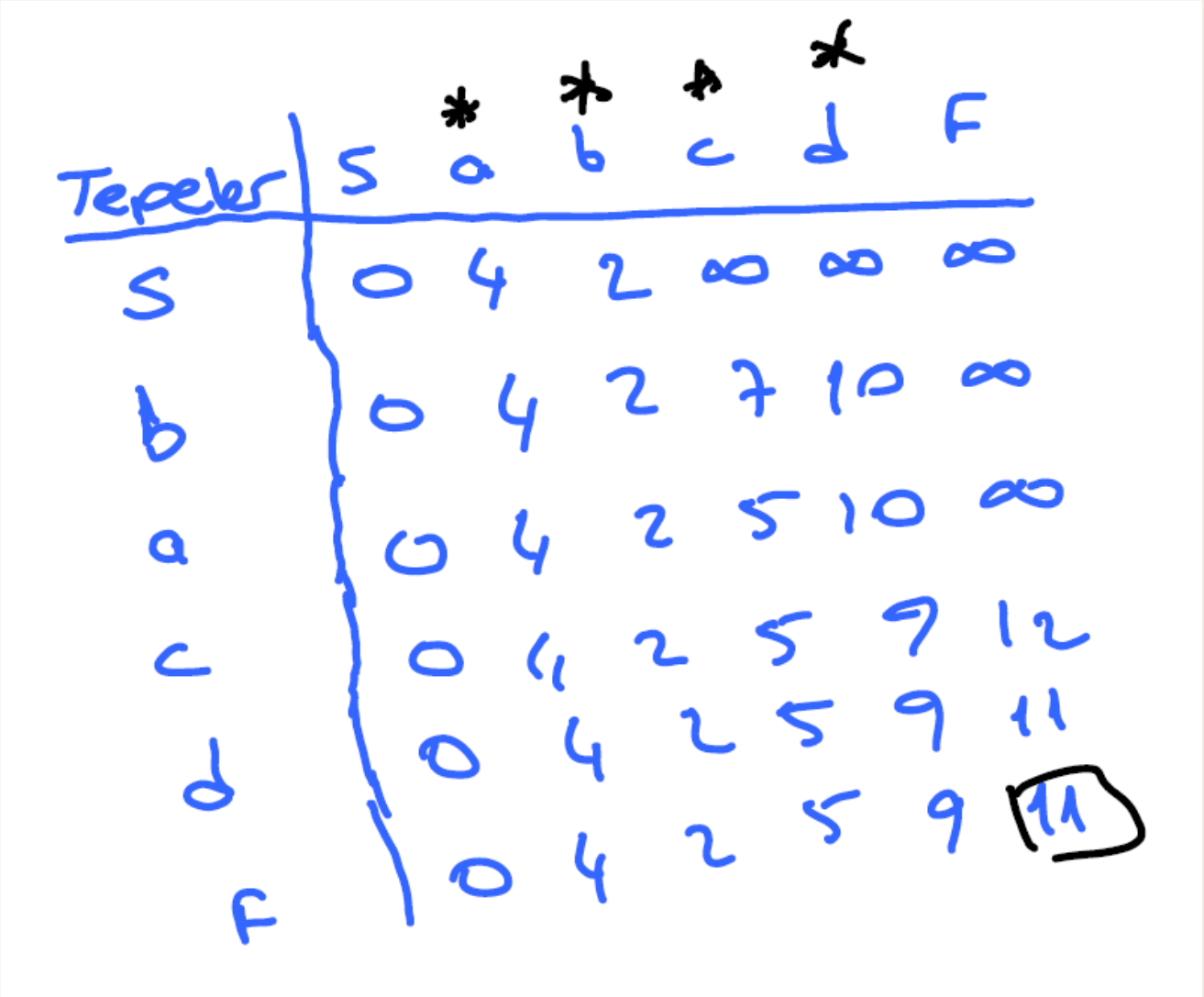


4. Adım sonrası yeni tablo aşağıdaki gibidir.



Tepeler	s	a*	b*	c*	d*	F
s	0	4	2	∞	∞	∞
b	0	4	2	7	10	∞
a	0	4	2	5	10	∞
c	0	4	2	5	9	12
d	0	4	2	5	9	11

5. adım: F son tepe F=11 için tablo aşağıdaki gibidir.



Tepeler	s	a*	b*	c*	d*	F
s	0	4	2	∞	∞	∞
b	0	4	2	7	10	∞
a	0	4	2	5	10	∞
c	0	4	2	5	9	12
d	0	4	2	5	9	11
F	0	4	2	5	9	11

S den diğer tüm tepelere en kısa uzaklıklar

Tepeler	S'den en kısa yollar	Min. uzaklık
a	S→a	4
b	S→b	2
c	S→a→c	5
d	S→a→x→d	9
F	S→a→c→d→F	11

Kaynaklar

- *1- Discrete Mathematics and Its Applications*, Kennet H. Rosen

(Ayrık Matematik ve Uygulamaları, Kennet H. Rosen (Türkçe çeviri), Palme yayıncılık)

- *2- Discrete Mathematics: Elementary and Beyond*, L. Lovász, J. Pelikán, K. Vesztergombi, 2003.
- *3- Introduction to Algorithms*, T.H. Cormen, C.E. Leiserson, R.L. Rivest, C. Stein, 2009.
- *4- Introduction To Design And Analysis Of Algorithms*, A. Levitin, 2008.
- *5-Algoritmalar (Teoriden Uygulamalara)*, Vasif V. Nabiyev, Seçkin Yayıncılık